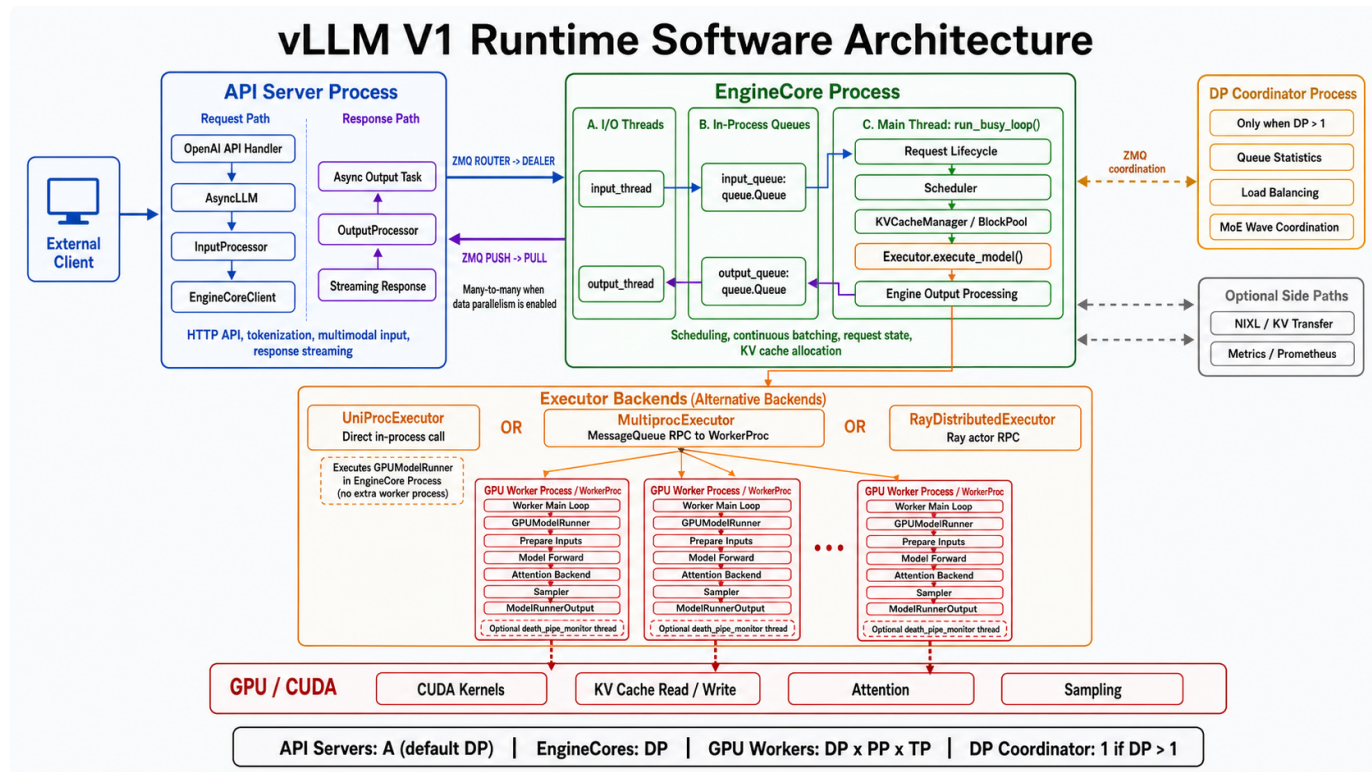


第2课 vLLM V1整体架构

v0.25.1

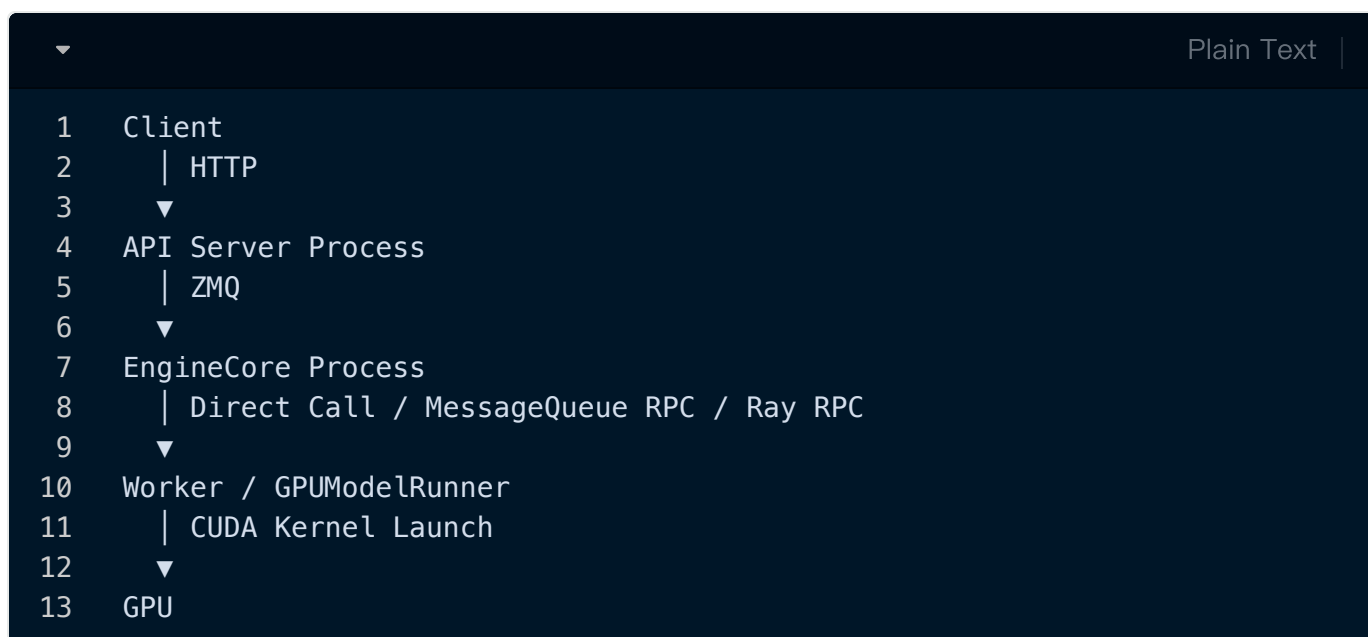
https://github.com/vllm-project/vllm/blob/main/docs/design/arch_overview.md



一个请求会跨越四层边界：**模块是代码职责边界，线程是并发边界，进程是内存隔离边界，GPU 是设备边界。**

整体边界

边界	通信双方	通信方式
网络	Client ↔ API Server	HTTP/gRPC
进程	API Server ↔ EngineCore	ZMQ
线程	EngineCore I/O Thread ↔ Main Thread	<code>queue.Queue</code>
进程, 可选	EngineCore ↔ WorkerProc	MessageQueue RPC
设备	GPUModelRunner ↔ GPU	CUDA kernel launch



这里要特别区分：

- `InputProcessor`、`Scheduler`、`KVCacheManager` 是模块。
- `input_thread`、EngineCore main thread、`output_thread` 是线程。
- API Server、EngineCore、WorkerProc 是进程。
- GPU 是独立的硬件执行设备。

模块之间不一定跨线程，线程之间不一定跨进程。

1. Client 请求进入 API Server

客户端通过 OpenAI Compatible API 发起请求：

```
1 POST /v1/chat/completions
```

API Server 负责：

- 解析 HTTP 和 OpenAI 请求格式
- 应用 chat template
- tokenize
- 处理图片等多模态输入
- 创建采样参数
- 把结果流式返回客户端

API Handler 最终调用 [AsyncLLM.generate\(\)](#)。

注意，这里主要运行在 API Server 的 `asyncio` 主线程中：

```
1 HTTP Handler coroutine
2   → AsyncLLM.generate() coroutine
3   → AsyncLLM.add_request() coroutine
```

它们是多个 `asyncio Task`，并不是每个请求创建一个操作系统线程。

2. InputProcessor 创建 EngineCoreRequest

在 `AsyncLLM.add_request()` 中，请求会经过 [InputProcessor.process_inputs\(\)](#)。

输入大致发生如下转换：

```
1 Prompt / Messages
2   + SamplingParams
3   + LoRA / multimodal metadata
4     |
5     ▼
6 EngineCoreRequest
```

`EngineCoreRequest` 是 API Server 与 EngineCore 之间的协议对象，包含：

- `request_id`
- `prompt_token_ids`
- sampling parameters
- multimodal inputs
- priority
- arrival time
- LoRA/KV transfer 等元数据

这里仍然没有发生进程通信，只是 API Server 进程内部的普通函数调用。

3. API Server 同时建立输出状态

创建 `EngineCoreRequest` 后，执行 `AsyncLLM._add_request()`：

```
1 self.output_processor.add_request(...)
2 await self.engine_core.add_request_async(request)
```

顺序非常重要：

1. 先在 `OutputProcessor` 中登记请求。
2. 再把请求发送给 EngineCore。

因此同一个逻辑请求，在 API Server 中会有一个输出侧状态：

```
1 OutputProcessor.request_states[request_id]
```

它负责后续的：

- detokenization
- stop string 检查
- logprobs 处理
- 流式输出状态
- 请求完成后的清理

4. EngineCoreClient 跨进程发送请求

`EngineCoreClient` 是 API Server 内部组件，不是单独进程。

入口是 `AsyncMPCClient.add_request_async()`，内部调用 `_send_input()`。

它把请求编码成：

```
1  RequestType.ADD
2  + msgpack(EngineCoreRequest)
3  + optional tensor buffers
```

然后通过 ZMQ 发送：

```
1  API Server Process      EngineCore Process
2  EngineCoreClient        input_thread
3  ZMQ ROUTER              → ZMQ DEALER
```

这是第一个真正的进程边界。

这里传递的是序列化数据。EngineCore 收到的对象不是 API Server 中原来的 Python 对象。

5. EngineCore input_thread 接收请求

EngineCore 的输入线程运行 `process_input_sockets()`。

它负责：

```
1  ZMQ recv_multipart()
2  → 判断 EngineCoreRequestType
3  → MsgpackDecoder.decode()
4  → preprocess_add_request()
5  → input_queue.put()
```

这里需要明确线程边界：

```
▼ Plain Text |
1  input_thread
2      |
3      | queue.Queue
4      ▼
5  EngineCore main thread
```

`input_thread` 只负责 I/O、反序列化和轻量预处理，不负责执行调度算法。

6. EngineCore 主线程接管请求

EngineCore 主线程运行 `run_busy_loop()`：

```
▼ Plain Text |
1  while running:
2      self._process_input_queue()
3      self._process_engine_step()
```

第一步从 `input_queue` 取出请求，并处理 `ADD`、`ABORT`、`UTILITY` 等消息。

新请求最终进入 `Scheduler.add_request()`：

```
▼ Plain Text |
1  EngineCoreRequest
2      → Request
3      → scheduler.requests[request_id]
4      → scheduler.waiting
```

从这里开始，请求的生命周期主要由 EngineCore 主线程拥有。

因此以下模块虽然职责不同，但主要由同一个线程调用：

```
1 Scheduler
2 Request queues
3 KVCacheManager
4 BlockPool
5 Executor
6 Engine output processing
```

这就是为什么 Scheduler 和 KV cache 管理通常不需要围绕请求状态添加大量线程锁。

7. Scheduler 生成本轮执行计划

每一次 EngineCore step 都调用 `Scheduler.schedule()`。

对于第一次执行的新请求：

```
1 waiting request
2   → prefix cache lookup
3   → 计算需要执行多少 token
4   → KVCacheManager.allocate_slots()
5   → 分配物理 KV blocks
6   → waiting → running
7   → 写入 SchedulerOutput
```

`SchedulerOutput` 不是模型输出，而是这一轮 GPU 的执行计划：

```
1 scheduled_new_reqs
2 scheduled_cached_reqs
3 num_scheduled_tokens
4 block_ids
5 scheduled_encoder_inputs
6 scheduled_spec_decode_tokens
7 preempted_req_ids
8 finished_req_ids
```

可以把它理解为：

Scheduler 不直接执行模型，而是生成“这一轮谁执行、执行多少 token、使用哪些 KV block”的计划。

8. EngineCore 调用 Executor

EngineCore 的 `step()` 中完成主流程：

```
1 scheduler_output = scheduler.schedule()
2 model_output = model_executor.execute_model(scheduler_output)
3 engine_core_outputs = scheduler.update_from_output(
4     scheduler_output,
5     model_output,
6 )
```

Executor 是一个统一抽象，但下面有三种不同边界。

UniProcExecutor

`UniProcExecutor.execute_model()` 使用进程内调用：

```
1 EngineCore
2   → UniProcExecutor
3   → Worker
4   → GPUModelRunner
```

不经过 Worker 子进程，也没有 MessageQueue RPC。

MultiprocExecutor

`MultiprocExecutor.execute_model()` 把执行命令发送给 `WorkerProc`：

```
1 EngineCore Process
2   → MessageQueue RPC
3   → WorkerProc Process
```


这是第二个进程边界，通常一个 GPU 对应一个 WorkerProc。

RayDistributedExecutor

[RayDistributedExecutor](#) 使用 Ray Actor RPC，适合分布式、多节点场景。

所以不能简单说：

EngineCore 一定通过进程通信调用 GPU Worker。

准确说法是：

是否跨进程取决于 Executor 实现。

9. GPUWorker 调用 GPUModelRunner

Worker 收到 `SchedulerOutput` 后，调用 [GPUWorker.execute_model\(\)](#)。

然后进入 [GPUModelRunner.execute_model\(\)](#)。

主要流程是：

```
1 SchedulerOutput
2   → 更新 Worker 侧请求状态
3   → 更新持久化 InputBatch
4   → 准备 input_ids / positions
5   → 准备 block_table / slot_mapping
6   → model forward
7   → attention backend
8   → sampling
9   → ModelRunnerOutput
```

这里还有一层设备边界：

```
1 GPUModelRunner (CPU Python/C++)
2   |
3   | CUDA kernel launch
4   ▼
5 GPU
```

GPU 负责：

- GEMM
- attention
- RMSNorm
- RoPE
- KV cache 读写
- sampling kernels

GPU 不是一个 vLLM 进程，也不会直接接收 `SchedulerOutput`。

10. 模型结果回到 EngineCore

GPUModelRunner 返回 `ModelRunnerOutput` 后，EngineCore 调用 [Scheduler.update_from_output\(\)](#)。

这里负责：

- 接收新 token
- 更新 `Request.output_token_ids`
- 更新 `num_computed_tokens`
- 处理 speculative decoding 结果
- 判断 EOS、长度限制和停止状态
- 释放已结束请求的 KV blocks
- 构造 `EngineCoreOutput`

数据变化为：

```
▼ Plain Text |
1  ModelRunnerOutput
2    → Scheduler request state update
3    → EngineCoreOutput
4    → EngineCoreOutputs
```

11. output_thread 跨进程返回结果

EngineCore 主线程把结果放入：

```
1 output_queue.put(...)
```

`process_output_sockets()` 运行在独立的 `output_thread` 中：

```
1 EngineCore main thread
2   |
3   | output_queue: queue.Queue
4   ▼
5 output_thread
6   |
7   | ZMQ PUSH → PULL
8   ▼
9 API Server EngineCoreClient
```

独立输出线程可以让序列化和 ZMQ 发送与下一轮调度、GPU 执行发生重叠。

12. API Server 完成 Detokenization

API Server 中的 `AsyncLLM.output_handler` 是后台 `asyncio Task`。

它不断执行：

```
1 engine_core.get_output_async()
2   → OutputProcessor.process_outputs()
3   → per-request RequestOutputCollector
```

`OutputProcessor.process_outputs()` 负责：

- token IDs 转换成文本
- stop string 检查
- logprobs 处理

- 构造 `RequestOutput`
- 把结果放进请求专属队列

最初的 `AsyncLLM.generate()` 从该队列中读取结果并 `yield`，API Handler 再通过 HTTP streaming 返回客户端。

同一请求在不同层的状态

同一个逻辑请求会在不同进程中存在不同表示：

	Plain Text
1	API Server Process
2	EngineCoreRequest
3	OutputProcessor.RequestState
4	RequestOutputCollector
5	
6	EngineCore Process
7	Scheduler.Request
8	waiting / running 状态
9	KVCacheManager.req_to_blocks
10	
11	WorkerProc Process
12	Cached request state
13	persistent InputBatch
14	block table
15	
16	GPU
17	input tensors
18	KV cache tensors
19	attention output

这些不是共享的同一个 Python 对象，而是不同层针对自己职责维护的状态。

课程总结

API Server 负责输入输出，EngineCore 负责请求生命周期、调度和 KV cache，Executor 负责屏蔽不同部署方式，Worker 与 GPUModelRunner 负责构造执行批次，GPU 负责数值计算。API Server 与 EngineCore 通过 ZMQ 隔离，EngineCore 的 I/O 线程与主线程通过 Python 队列衔接，EngineCore 与 Worker 是否跨进程则由 Executor 类型决定。

